

ME 314 Final Project - Simulation of a jack in a cup

Submitted by - Phani Kiran Vadlamani

The default project has been chosen for the final project of Machine Dynamics Course

Description

The default project simulates a scenario where a jack is shaken inside a cup. This involves the interaction of two rigid bodies, with the cup experiencing external forces and torques that cause the jack to bounce within it. The entire system is subjected to gravity.

System Description

The system state variables are defined as

$$q = \begin{bmatrix} x_C \\ y_C \\ \theta_C \\ x_J \\ y_J \\ \theta_J \end{bmatrix}$$

where

x_C is the x -coordinate of COM of the cup in world frame

y_C is the y -coordinate of COM of the cup in world frame

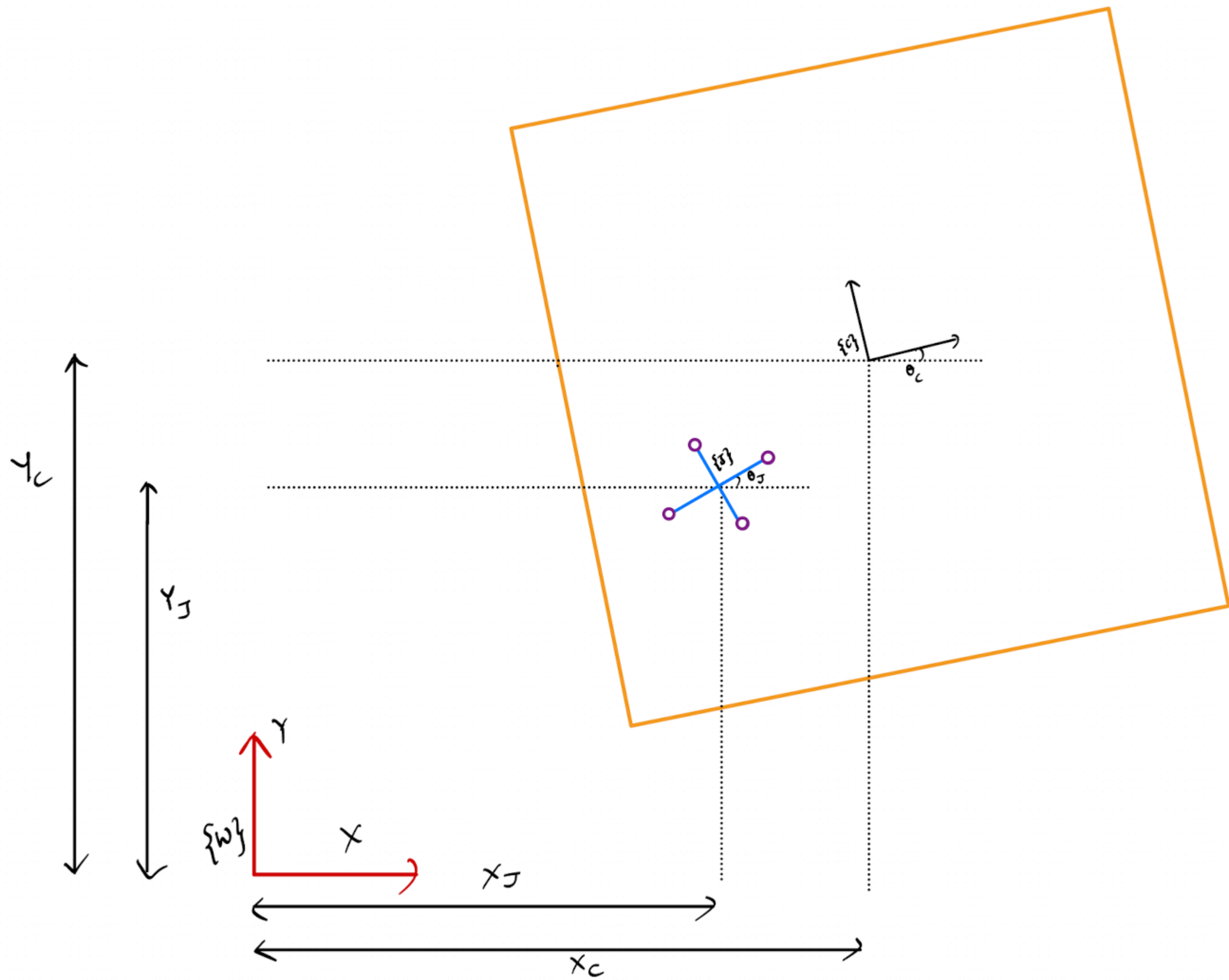
θ_C is the angle made by x -axis cup frame C with the x -axis of W frame

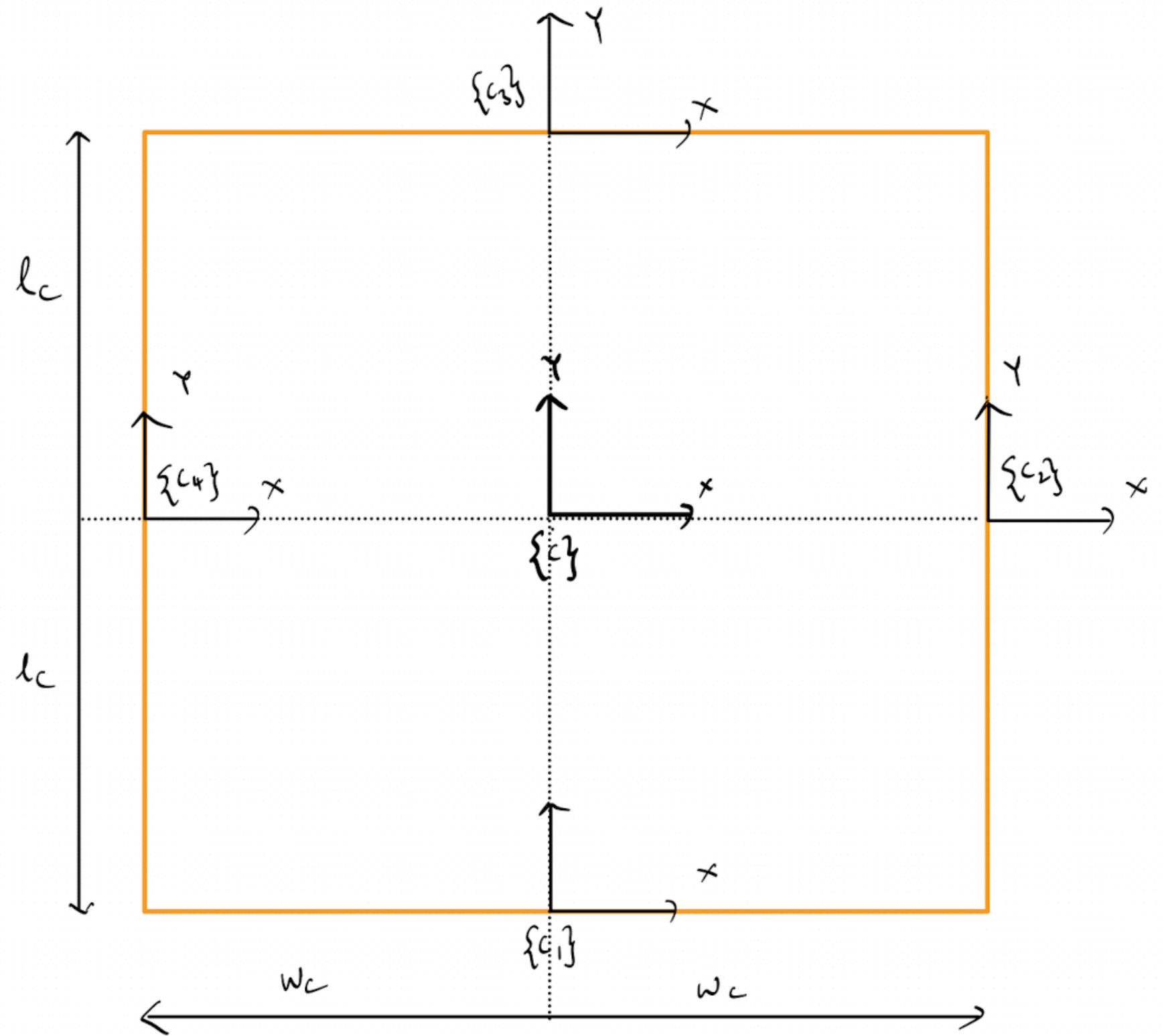
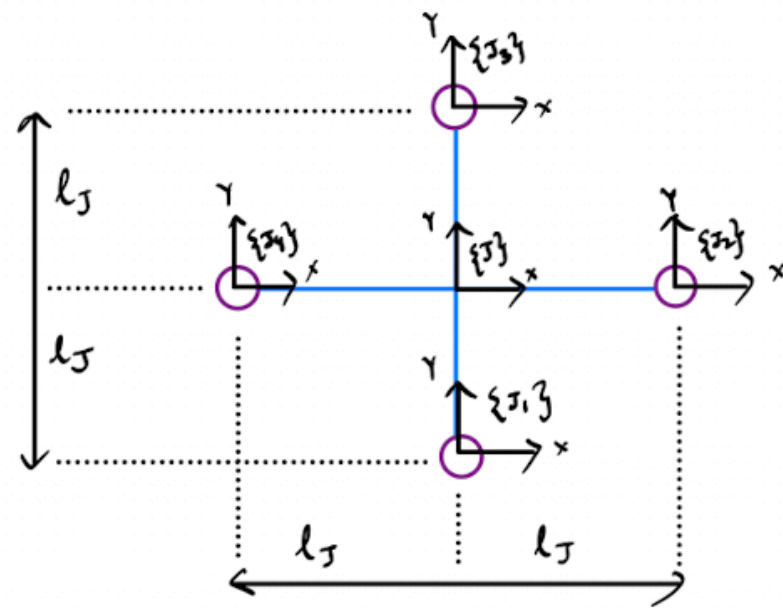
x_J is the x -coordinate of COM of the jack in world frame

y_J is the y -coordinate of COM of the jack in world frame

θ_J is the angle made by x -axis jack frame J with the x -axis of W frame

The frames W , C and J are shown in the below diagram:





Rigid Body Transformations

Transformation between the world frame and cup frame

$$g_{WJ} = \begin{bmatrix} \cos \theta_J & -\sin \theta_J & 0 & x_J \\ \sin \theta_J & \cos \theta_J & 0 & y_J \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Transformation between the world frame and jack frame

$$g_{WC} = \begin{bmatrix} \cos \theta_C & -\sin \theta_C & 0 & x_C \\ \sin \theta_C & \cos \theta_C & 0 & y_C \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Transformations between the frames defined in the cup

$$g_{CC_1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -l_C \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad g_{CC_2} = \begin{bmatrix} 1 & 0 & 0 & w_C \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$g_{CC_3} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & l_C \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad g_{CC_4} = \begin{bmatrix} 1 & 0 & 0 & -w_C \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Transformations between the frames defined in the jack

$$g_{JJ_1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -l_J \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad g_{JJ_2} = \begin{bmatrix} 1 & 0 & 0 & l_J \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$g_{JJ_3} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & l_J \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad g_{JJ_4} = \begin{bmatrix} 1 & 0 & 0 & -l_J \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Required transformations to cup walls from the world frame can be calculated as follows:

$$g_{WC_i} = g_{WC} g_{CC_i}$$

Similarly the transformations to the jack individual masses from the world frame can be calculated as follows:

$$g_{WJ_i} = g_{WJ} g_{JJ_i}$$

where $i = 1, 2, 3 \text{ and } 4$

Modeling choice of the cup and jack

The cup is modeled as a rectangular box with each wall being a 1D line. The dimensions of the cup are $2l_C$ and $2w_C$. The mass of the cup is m_C

The jack is modeled as a point masses of mass m_J . The masses 1, 2, 3 and 4 are connected in a cross-shaped configuration using massless one-dimensional rods. The length of each rod with measured from the centre of jack to the each mass is l_J . The mass of the jack is m_J

Rotational Inertia

Cup is modeled as 1-dimensional lines, which are arranged in a rectangle with uniform linear density. The mass of each wall is defined to be

$$m_{wall_i} = \frac{\text{length of wall}_i}{2l_C + 2w_C} m_C$$

The 6×6 mass matrix for the cup in the cup frame is given by

$$I_C = \begin{bmatrix} m_C & 0 & 0 & 0 & 0 & 0 \\ 0 & m_C & 0 & 0 & 0 & 0 \\ 0 & 0 & m_C & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{4l_C^3 m_C}{3(4l_C+4w_C)} + \frac{4l_C^2 m_C w_C}{4l_C+4w_C} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{4l_C m_C w_C^2}{4l_C+4w_C} + \frac{4m_C w_C^3}{3(4l_C+4w_C)} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{4l_C^3 m_C}{3(4l_C+4w_C)} + \frac{4l_C^2 m_C w_C}{4l_C+4w_C} + \frac{4l_C m_C w_C^2}{4l_C+4w_C} + \frac{4m_C w_C^3}{3(4l_C+4w_C)} \end{bmatrix}$$

Jack is modeled as 4 point masses, connected by massless rigid rods. The mass of each point is defined to be

$$m_{J_i} = \frac{m_J}{4}$$

The 6×6 mass matrix for the jack in the jack frame is given by

$$I_J = \begin{bmatrix} m_J & 0 & 0 & 0 & 0 & 0 \\ 0 & m_J & 0 & 0 & 0 & 0 \\ 0 & 0 & m_J & 0 & 0 & 0 \\ 0 & 0 & 0 & 2l_J^2 m_J & 0 & 0 \\ 0 & 0 & 0 & 0 & 2l_J^2 m_J & 0 \\ 0 & 0 & 0 & 0 & 0 & 4l_J^2 m_J \end{bmatrix}$$

Modeling the External Forces and Torque

To shake the cup, a force is applied in x and y direction (world frame) and a torque in z axis (world frame). The forcing function is taken as the product of a constant and the difference of the actual desired coordinates from the actual coordinates (Similar to Proportional Control).

For the x direction, the force is given as:

$$F_{x_C} = K_{p_{x_C}} [x_{C,des} - x_C]$$

$$x_{C,des} = A_{x_C} \cos(2\pi f_{x_C} t)$$

where $K_{p_{x_C}}$ is proportional gain constant, A_{x_C} is amplitude of oscillation, and f_{x_C} is frequency of oscillation.

For the y direction, the force is given as (a term for gravity compensation is also included):

$$F_{y_C} = m_C g + K_{p_{y_C}} [y_{C,des} - y_C]$$

$$y_{C,des} = A_{y_C} \cos(2\pi f_{y_C} t)$$

where $K_{p_{y_C}}$ is proportional gain constant, A_{y_C} is amplitude of oscillation, and f_{y_C} is frequency of oscillation.

To bring a variation in θ_C , torque is applied in in z-axis to drive θ_C to 0:

$$\tau_C = -K_{p_{\theta_C}} \theta_C$$

The parameters (Proportional gains and frequency) have been set to create a sufficiently chaotic motion of the cup.

Lagrangian

The Lagrangian is defined as:

$$L = [KE_C + KE_J] - [PE_C + PE_J]$$

Kinetic energy of the cup is given by:

$$KE_C = \frac{1}{2}\mathbf{v}_C^T I_C \mathbf{v}_C$$

where $\mathbf{v}_C$ is the body twist of cup expressed in cup frame and I_C is the 6×6 mass matrix

Kinetic energy of the jack is given by:

$$KE_J = \frac{1}{2}\mathbf{v}_J^T I_J \mathbf{v}_J$$

where $\mathbf{v}_J$ is the body twist of jack expressed in jack frame and I_J is the 6×6 mass matrix

Potential energy of the cup is given by:

$$PE_C = m_C g \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix} g_{WC} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Potential energy of the jack is given by:

$$PE_J = m_J g \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix} g_{WJ} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Euler-Lagrangian Equations

The Euler-Lagrange Equations are defined as:

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}} \right) - \frac{\partial L}{\partial q} = F_{ext}$$

$$F_{ext} = \begin{bmatrix} F_{x_C} \\ F_{y_C} \\ \tau_c \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Impact Conditions

Impact conditions ϕ_{ij} are defined using transformations.

$$\phi_{ij} = \text{impact_direction} \cdot g_{C_iW_j} \cdot \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}^\top$$

where i indicates the wall i and j indicates the vertex j of the jack both vary from 1 to 4 and impact_direction takes the value

$$\begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix}$$

and

$$\begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}$$

when $i = 1, 3$ and $i = 2, 4$ respectively

It is important to note that for 8 impact equations the body is not in impact when $\phi_{ij} > 0$ and for the other 8, the body is not in impact when $\phi_{ij} < 0$. The sign change is tracked for all 16 impacts to determine whether the body is experiencing an impact or not.

The actual calculated symbolic impact conditions can be seen as outputs of the code.

Impact updates

Impact updates can be applied by defining the Hamiltonian:

$$H = \frac{dL}{d\dot{q}}\dot{q} - L$$

The Hamiltonian must be conserved

$$H \Big|_{\tau^-}^{\tau^+} = 0$$

The rest of the equations needed to solve for the impact update are:

$$\frac{dL}{d\dot{q}} \Big|_{\tau^-}^{\tau^+} = \lambda \frac{\partial \phi}{\partial q}$$

To apply an update, $\lambda, \dot{x}_C(\tau^+), \dot{y}_C(\tau^+), \dot{\theta}_C(\tau^+), \dot{x}_J(\tau^+), \dot{y}_J(\tau^+)$, and $\dot{\theta}_J(\tau^+)$ were solved for in this system of seven equations using the state before the impact occurred and holding positions constant during impact.

Impact Evauation

- Impact updates are triggered whenever any of the 16 impact conditions undergo a sign change.
- Upon detecting an impact, the simulation rewinds by one time step, and the velocities of the configuration variables are updated while keeping the positions unchanged.
- In cases of simultaneous impacts, only the first detected impact is considered, and the system variables are updated accordingly.
- An alternative approach involves randomly selecting one impact from the detected impacts for updating the system.

Observations

- The simulation output behaves as expected, showing oscillation of the cup in the x and y directions. The cup does not fall due to gravity and tends to return to ($\theta_C = 0$).
- The oscillation of the cup is imperfect (not sinusoidal in nature) due to impacts with the jack.
- The jack's movement is accurate during single impacts, but during simultaneous impacts, the behavior becomes inaccurate. This is expected since only one impact is evaluated at a time.
- The jack exhibits "bouncing" and "rotation" when it impacts the cup.

- The simulation is highly sensitive to parameters such as proportional gains, amplitude, frequency, and the simulation time step. Unrealistically high velocities may cause the jack to leave the cup and the simulation fails to recover.
- This issue could be mitigated by reducing the simulation time step, but this significantly increases computation time. Additionally, the current strategy of considering only one impact during simultaneous impacts is suboptimal and could be improved.

```
In [1]: #####
# If you're using Google Colab, uncomment this section by selecting the whole section and press
# ctrl+'/' on your and keyboard. Run it before you start programming, this will enable the nice
# LaTeX "display()" function for you. If you're using the local Jupyter environment, leave it alone
#####
import sympy as sym
import numpy as np
import matplotlib.pyplot as plt

def custom_latex_printer(exp,**options):
    from google.colab.output._publish import javascript
    url = "https://cdnjs.cloudflare.com/ajax/libs/mathjax/3.1.1/latest.js?config=TeX-AMS_HTML"
    javascript(url=url)
    return sym.printing.latex(exp,**options)
sym.init_printing(use_latex="mathjax", latex_printer=custom_latex_printer)
```

Helper Functions

```
In [2]: # Helper Functions
def theta_p_to_SE3(theta, p):
    """
    Create an SE(3) transformation matrix from a rotation angle and translation vector.
    """

    g = sym.Matrix([
        [sym.cos(theta), -sym.sin(theta), 0.0, p[0]],
        [sym.sin(theta),  sym.cos(theta), 0.0, p[1]],
        [0.0, 0.0, 1.0, 0.0],
        [0.0, 0.0, 0.0, 1.0]
    ])

    return g

def SE3_to_rot_p(g):
    """
    Extract the rotation matrix (R) and translation vector (p) from an SE(3) matrix.
    """

    return g[0:3, 0:3], g[0:3,3]

def SE3_inv(g):
    """
    Compute the inverse of an SE(3) transformation matrix.
    """

    R, p = SE3_to_rot_p(g)
    R_transpose = R.transpose()
    g_inv = sym.Matrix.vstack(sym.Matrix.hstack(R_transpose, -R_transpose*p), \
                               sym.Matrix([[0.0, 0.0, 0.0, 1.0]]))

    return g_inv

def vec_to_so3(v):
    """
    Convert a 3D angular velocity vector into its skew-symmetric matrix form (so(3)).
    """

    sym
    return sym.Matrix([
        [0, -v[2], v[1]],
        [v[2], 0, -v[0]],
        [v[0], v[1], 0]
```



```

    [-v[1], v[0], 0]
])

def vec_to_se3(v):
    """
    Convert a 6D spatial velocity vector into its se(3) matrix form.
    """
    g_dot = sym.Matrix.vstack( \
        sym.Matrix.hstack(vec_to_so3(sym.Matrix([v[3], v[4], v[5]])), \
            [[v[0]], [v[1]], [v[3]]]), \
        sym.Matrix([[0.0, 0.0, 0.0, 0.0]])
    )
    return g_dot

def se3_to_vec(g_dot):
    """
    Convert an se(3) matrix into its 6D vector representation.
    """
    v = sym.Matrix([g_dot[0, 3], g_dot[1, 3], g_dot[2, 3], g_dot[2, 1], g_dot[0, 2], g_dot[1, 0]])
    return v

def S03_dot_body(R, omega):
    """
    Compute the time derivative of an S0(3) rotation matrix in the body frame.
    """
    omega_hat = vec_to_so3(omega)
    return R*omega_hat

def SE3_dot_body(R, omega, p_dot):
    """
    Compute the time derivative of an SE(3) transformation matrix in the body frame.
    """
    R_dot = S03_dot_body(R, omega)
    g_dot = sym.Matrix.vstack(sym.Matrix.hstack(R_dot, p_dot), \
        sym.Matrix([[0.0, 0.0, 0.0, 0.0]]))

    return g_dot

def normalize_angles(angles):
    """
    Normalize angles in a NumPy array to the range (-pi, pi]
    """
    return np.arctan2(np.sin(angles), np.cos(angles))

```

Declare states and constants of the system

```

In [3]: from sympy.abc import t

# Define the constants
l_J, l_C, w_C, m_J, m_C, g = sym.symbols('l_J, l_C, w_C, m_J, m_C, g')

# Define the system states

# State of the jack x_J, y_J and theta_J
x_J = sym.Function(r'x_J')(t)
y_J = sym.Function(r'y_J')(t)
theta_J = sym.Function(r'\theta_J')(t)

# State of the cup x_C, y_C and theta_C
x_C = sym.Function(r'x_C')(t)
y_C = sym.Function(r'y_C')(t)
theta_C = sym.Function(r'\theta_C')(t)

```

```

# Moment of Inertia of the jack in the body frame of jack
I_J1_x = 0
I_J2_x = m_J * l_J**2
I_J3_x = 0
I_J4_x = I_J2_x

I_J1_y = m_J * l_J**2
I_J2_y = 0
I_J3_y = I_J1_y
I_J4_y = 0

I_J1_z = m_J * l_J**2
I_J2_z = I_J1_z
I_J3_z = I_J1_z
I_J4_z = I_J1_z

I_J_x = I_J1_x + I_J2_x + I_J3_x + I_J4_x
I_J_y = I_J1_y + I_J2_y + I_J3_y + I_J4_y
I_J_z = I_J1_z + I_J2_z + I_J3_z + I_J4_z

# Moment of Inertia for the cup in the body frame of cup
I_C1_x = (m_C * (2*w_C)/(4*l_C + 4*w_C)) * l_C**2
I_C2_x = (m_C * (2*l_C)/(4*l_C + 4*w_C)) * (2*l_C)**2 / 12
I_C3_x = I_C1_x
I_C4_x = I_C2_x

I_C1_y = (m_C * (2*w_C)/(4*l_C + 4*w_C)) * (2*w_C)**2 / 12
I_C2_y = (m_C * (2*l_C)/(4*l_C + 4*w_C)) * w_C**2
I_C3_y = I_C1_y
I_C4_y = I_C2_y

I_C1_z = (m_C * (2*w_C)/(4*l_C + 4*w_C)) * (2*w_C)**2 / 12 + (m_C * (2*w_C)/(4*l_C + 4*w_C)) * l_C**2
I_C2_z = (m_C * (2*l_C)/(4*l_C + 4*w_C)) * (2*l_C)**2 / 12 + (m_C * (2*l_C)/(4*l_C + 4*w_C)) * w_C**2
I_C3_z = I_C1_z
I_C4_z = I_C2_z

I_C_x = I_C1_x + I_C2_x + I_C3_x + I_C4_x
I_C_y = I_C1_y + I_C2_y + I_C3_y + I_C4_y
I_C_z = I_C1_z + I_C2_z + I_C3_z + I_C4_z

# Define the 6x6 mass matrix
# Note that 3 elements of the matrix (m in z direction, J in x and y direction)
# should not have any impact because we are dealing with 2D rigid body motions
I_J = sym.diag(m_J, m_J, m_J, I_J_x, I_J_y, I_J_z)
I_C = sym.diag(m_C, m_C, m_C, I_C_x, I_C_y, I_C_z)

print('\033[1mI_J:')
display(I_J)
print("\n")

print('\033[1mI_C:')
display(I_C)

```

I_J:

$$\begin{bmatrix} m_J & 0 & 0 & 0 & 0 & 0 \\ 0 & m_J & 0 & 0 & 0 & 0 \\ 0 & 0 & m_J & 0 & 0 & 0 \\ 0 & 0 & 0 & 2l_J^2 m_J & 0 & 0 \\ 0 & 0 & 0 & 0 & 2l_J^2 m_J & 0 \\ 0 & 0 & 0 & 0 & 0 & 4l_J^2 m_J \end{bmatrix}$$

I_C:

$$\begin{bmatrix} m_C & 0 & 0 & 0 & 0 & 0 \\ 0 & m_C & 0 & 0 & 0 & 0 \\ 0 & 0 & m_C & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{4l_C^3 m_C}{3(4l_C+4w_C)} + \frac{4l_C^2 m_C w_C}{4l_C+4w_C} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{4l_C m_C w_C^2}{4l_C+4w_C} + \frac{4m_C w_C^3}{3(4l_C+4w_C)} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{4l_C^3 m_C}{3(4l_C+4w_C)} + \frac{4l_C^2 m_C w_C}{4l_C+4w_C} + \frac{4l_C m_C w_C^2}{4l_C+4w_C} + \frac{4m_C w_C^3}{3(4l_C+4w_C)} \end{bmatrix}$$

Declare transformation matrices

```
In [4]: g_wJ = theta_p_to_SE3(theta_J, sym.Matrix([x_J, y_J, 0]))
g_wC = theta_p_to_SE3(theta_C, sym.Matrix([x_C, y_C, 0]))

print('\033[1mg_wJ:')
display(g_wJ)
print("\n")

print('\033[1mg_wC:')
display(g_wC)
```

g_wJ:

$$\begin{bmatrix} \cos(\theta_J(t)) & -\sin(\theta_J(t)) & 0.0 & x_J(t) \\ \sin(\theta_J(t)) & \cos(\theta_J(t)) & 0.0 & y_J(t) \\ 0.0 & 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

g_wC:

$$\begin{bmatrix} \cos(\theta_C(t)) & -\sin(\theta_C(t)) & 0.0 & x_C(t) \\ \sin(\theta_C(t)) & \cos(\theta_C(t)) & 0.0 & y_C(t) \\ 0.0 & 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

Compute body twists of jack and cup

```
In [5]: V_J = se3_to_vec(SE3_inv(g_wJ) * SE3_dot_body(g_wJ[0:3, 0:3], [0, 0, sym.diff(theta_J)], g_wJ[0:3, 3].diff(t)))
V_C = se3_to_vec(SE3_inv(g_wC) * SE3_dot_body(g_wC[0:3, 0:3], [0, 0, sym.diff(theta_C)], g_wC[0:3, 3].diff(t)))

print('\033[1mBody twist of the jack - V_J:')
display(V_J)
print("\n")
print('\033[1mBody twist of the cup - V_C:')
display(V_C)
```

Body twist of the jack - V_J:

$$\begin{bmatrix} \sin(\theta_J(t))\frac{d}{dt}y_J(t) + \cos(\theta_J(t))\frac{d}{dt}x_J(t) \\ -\sin(\theta_J(t))\frac{d}{dt}x_J(t) + \cos(\theta_J(t))\frac{d}{dt}y_J(t) \\ 0 \\ 0 \\ 0 \\ \sin^2(\theta_J(t))\frac{d}{dt}\theta_J(t) + \cos^2(\theta_J(t))\frac{d}{dt}\theta_J(t) \end{bmatrix}$$

Body twist of the cup - V_C:

$$\begin{bmatrix} \sin(\theta_C(t))\frac{d}{dt}y_C(t) + \cos(\theta_C(t))\frac{d}{dt}x_C(t) \\ -\sin(\theta_C(t))\frac{d}{dt}x_C(t) + \cos(\theta_C(t))\frac{d}{dt}y_C(t) \\ 0 \\ 0 \\ 0 \\ \sin^2(\theta_C(t))\frac{d}{dt}\theta_C(t) + \cos^2(\theta_C(t))\frac{d}{dt}\theta_C(t) \end{bmatrix}$$

Compute body KE and PE of the jack and the cup. Then calculate lagrangian of the system

```
In [6]: # Compute KE of Jack and Cup
KE_J = (0.5 * (V_J.T) * I_J * (V_J))[0]
KE_C = (0.5 * (V_C.T) * I_C * (V_C))[0]

# display(sym.simplify(KE1))
# display(sym.simplify(KE2))

# Compute the PE of leg 1 and leg 2
PE_J = m_J * g * (sym.Matrix([[0, 1, 0, 0]]) * g_wJ * (sym.Matrix([[0, 0, 0, 1]]).T))[0]
PE_C = m_C * g * (sym.Matrix([[0, 1, 0, 0]]) * g_wC * (sym.Matrix([[0, 0, 0, 1]]).T))[0]

# display(PE1)
# display(PE2)

KE = KE_J + KE_C
PE = PE_J + PE_C

# Compute the lagrangian of the system
lagrangian = KE - PE
lagrangian = sym.simplify(lagrangian)

# Display Lagrangian
print('\033[1mLagrangian of the system: ')
display(lagrangian)
print("\n")\

# Wrap the lagrangian into a SymPy Matrix
L = sym.Matrix([lagrangian])
# display(L)
```

Lagrangian of the system:

$$\frac{0.1666666666666667m_C \left(l_C^3 + 3l_Cw_C(l_C + w_C) + w_C^3\right) \left(\frac{d}{dt}\theta_C(t)\right)^2 + (l_C + w_C) \left(-1.0gm_Cy_C(t) - 1.0gm_Jy_J(t) + 2.0l_J^2m_J\left(\frac{d}{dt}\theta_J(t)\right)^2 + 0.5m_C\left(\frac{d}{dt}x_C(t)\right)^2 + 0.5m_C\left(\frac{d}{dt}y_C(t)\right)^2 + 0.5m_J\left(\frac{d}{dt}x_J(t)\right)^2 + 0.5m_J\left(\frac{d}{dt}y_J(t)\right)^2\right)}{l_C + w_C}$$

Compute the LHS of EL equations

```
In [7]: q = sym.Matrix([x_C, y_C, theta_C, x_J, y_J, theta_J])
qdot = q.diff(t)
qddot = qdot.diff(t)

# Compute Jacobian
dLdq = L.jacobian(q).T
dLdqdot = L.jacobian(qdot).T

# print('\033[1mEuler Lagrangian Equations')
EL_lhs = dLdqdot.diff(t) - dLdq
```

```
print('\033[1mLHS of the EL equations:')
display(EL_lhs)
```

LHS of the EL equations:

$$\begin{bmatrix} 1.0m_C \frac{d^2}{dt^2} x_C(t) \\ 1.0gm_C + 1.0m_C \frac{d^2}{dt^2} y_C(t) \\ \frac{0.3333333333333333m_C \left(l_C^3 + 3l_Cw_C(l_C + w_C) + w_C^3 \right) \frac{d^2}{dt^2} \theta_C(t)}{l_C + w_C} \\ 1.0m_J \frac{d^2}{dt^2} x_J(t) \\ 1.0gm_J + 1.0m_J \frac{d^2}{dt^2} y_J(t) \\ 4.0l_J^2m_J \frac{d^2}{dt^2} \theta_J(t) \end{bmatrix}$$

Compute the RHS of EL Equations

```
In [8]: # Define the constants
f_x_C, f_y_C, K_p_x_C, K_p_y_C, K_p_theta_C, A_x_C, A_y_C = sym.symbols('f_{x_C}, f_{y_C}, K_{p_{x_C}}, K_{p_{y_C}}, K_{p_{theta_C}}, A_{x_C}, A_{y_C}')

# External forces to shake the cup
x_C_d = A_x_C * sym.cos(2 * sym.pi * f_x_C * t)
y_C_d = A_y_C * sym.cos(2 * sym.pi * f_y_C * t)
theta_C_d = 0

print('\033[1mExternal Forces on the cup:')
F_x_C = K_p_x_C * (x_C_d - x_C)
F_y_C = m_C * g + K_p_y_C * (y_C_d - y_C)
F_theta_C = K_p_theta_C * (theta_C_d - theta_C)

print('\033[1mX:')
display(F_x_C)
print("\n")

print('\033[1mY:')
display(F_y_C)
print("\n")

print('\033[1mTheta:')
display(F_theta_C)
print("\n")

EL_rhs = sym.Matrix([[F_x_C, F_y_C, F_theta_C, 0, 0, 0]]).T
print('\033[1mRHS of the EL equations:')
display(EL_rhs)
```

External Forces on the cup:

X:

$$K_{p_{x_C}} \left(A_{x_C} \cos \left(2\pi f_{x_C} t \right) - x_C(t) \right)$$

Y:

$$K_{p_{y_C}} \left(A_{y_C} \cos \left(2\pi f_{y_C} t \right) - y_C(t) \right) + gm_C$$

Theta:

$$-K_{p_{theta_C}} \theta_C(t)$$

RHS of the EL equations:

$$\begin{bmatrix} K_{p_{x_C}} \left(A_{x_C} \cos \left(2\pi f_{x_C} t \right) - x_C(t) \right) \\ K_{p_{y_C}} \left(A_{y_C} \cos \left(2\pi f_{y_C} t \right) - y_C(t) \right) + gm_C \\ -K_{p_{\theta_C}} \theta_C(t) \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

```
In [9]: EL_eqns = sym.Eq(sym.Matrix(EL_lhs), sym.Matrix(EL_rhs))
EL_solns = sym.solve(EL_eqns, qddot)

print('\033[1mEL equations solved:')
for qddot_ in qddot:
    display(sym.Eq(qddot_, sym.simplify(EL_solns[qddot_])))
```

EL equations solved:

$$\frac{d^2}{dt^2}x_C(t) = \frac{K_{p_{x_C}} \left(A_{x_C} \cos \left(6.28318530717959 f_{x_C} t \right) - x_C(t) \right)}{m_C}$$

$$\frac{d^2}{dt^2}y_C(t) = \frac{K_{p_{y_C}} \left(A_{y_C} \cos \left(6.28318530717959 f_{y_C} t \right) - y_C(t) \right)}{m_C}$$

$$\frac{d^2}{dt^2}\theta_C(t) = -\frac{3.0K_{p_{\theta_C}} \theta_C(t)}{m_C \left(l_C^2 + 2.0l_Cw_C + w_C^2 \right)}$$

$$\frac{d^2}{dt^2}x_J(t) = 0.0$$

$$\frac{d^2}{dt^2}y_J(t) = -g$$

$$\frac{d^2}{dt^2}\theta_J(t) = 0.0$$

Substitute constants

```
In [10]: def sub_constants(eqn):
        """
        Substitute constant values into the given equation.
        """
        return eqn.subs([
            [m_J, 1],          # [kg] Mass of the jack
            [l_J, 1],          # [m] Length of the jack
            [m_C, 3],          # [kg] Mass of the cup
            [l_C, 3],          # [m] Length of the cup
            [w_C, 2],          # [m] Width of the cup
            [g, 9.8],          # [m/s^2] Acceleration due to gravity
            [f_x_C, 1.5],      # [1/s] Frequency of x oscillation for the cup
            [f_y_C, 1.5],      # [1/s] Frequency of y oscillation for the cup
            [A_x_C, 2],        # [m] Amplitude of x oscillation for the cup
            [A_y_C, 3],        # [m] Amplitude of y oscillation for the cup
            [K_p_x_C, 10.0],    # [N/m] Proportional gain for cup x position
            [K_p_y_C, 10.0],    # [N/m] Proportional gain for cup y position
            [K_p_theta_C, 50.0],# [Nm] Proportional gain for cup orientation
        ])
```

```
In [11]: state_dynamics_func = []
for qddot_ in qddot:
    state_dynamics_func.append(sym.lambdify(sym.Matrix([q, qdot, t]), sub_constants(EL_solns[qddot_])))
```


Impact Equations

```
In [12]: x_C_minus, y_C_minus, theta_C_minus = sym.symbols(r'x_C^-, y_C^-, \theta_C^-')
x_J_minus, y_J_minus, theta_J_minus = sym.symbols(r'x_J^-, y_J^-, \theta_J^-')

x_dot_C_minus, y_dot_C_minus, theta_dot_C_minus = sym.symbols(r'\dot{x}_C^-, \dot{y}_C^-, \dot{\theta}_C^-')
x_dot_J_minus, y_dot_J_minus, theta_dot_J_minus = sym.symbols(r'\dot{x}_J^-, \dot{y}_J^-, \dot{\theta}_J^-')

x_dot_C_plus, y_dot_C_plus, theta_dot_C_plus = sym.symbols(r'\dot{x}_C^+, \dot{y}_C^+, \dot{\theta}_C^+')
x_dot_J_plus, y_dot_J_plus, theta_dot_J_plus = sym.symbols(r'\dot{x}_J^+, \dot{y}_J^+, \dot{\theta}_J^+')

lam = sym.symbols(r'\lambda')

subs_minus = {
    q[0]:x_C_minus,
    q[1]:y_C_minus,
    q[2]:theta_C_minus,
    q[3]:x_J_minus,
    q[4]:y_J_minus,
    q[5]:theta_J_minus,
    qdot[0]:x_dot_C_minus,
    qdot[1]:y_dot_C_minus,
    qdot[2]:theta_dot_C_minus,
    qdot[3]:x_dot_J_minus,
    qdot[4]:y_dot_J_minus,
    qdot[5]:theta_dot_J_minus
}

subs_plus = {
    q[0]:x_C_minus,
    q[1]:y_C_minus,
    q[2]:theta_C_minus,
    q[3]:x_J_minus,
    q[4]:y_J_minus,
    q[5]:theta_J_minus,
    qdot[0]:x_dot_C_plus,
    qdot[1]:y_dot_C_plus,
    qdot[2]:theta_dot_C_plus,
    qdot[3]:x_dot_J_plus,
    qdot[4]:y_dot_J_plus,
    qdot[5]:theta_dot_J_plus
}

# display(subs_minus)
# display(subs_plus)

# Compute momentum at tau- and tau+
dLdqdot_minus = dLdqdot.subs(subs_minus)
dLdqdot_plus = dLdqdot.subs(subs_plus)

# Define the hamiltonian
hamiltonian = sym.simplify(dLdqdot.T.dot(qdot)-lagrangian)
# display(hamiltonian)

# Compute hamiltonian at tau- and tau+
hamiltonian_minus = hamiltonian.subs(subs_minus)
hamiltonian_plus = hamiltonian.subs(subs_plus)

impact_conditions = []
impact_equations = []

n_jack_vertices = 4
```

```

n_cup_walls = 4

g_wC1 = g_wC * theta_p_to_SE3(0, sym.Matrix([0, -l_C, 0]))
g_wC2 = g_wC * theta_p_to_SE3(0, sym.Matrix([w_C, 0, 0]))
g_wC3 = g_wC * theta_p_to_SE3(0, sym.Matrix([0, l_C, 0]))
g_wC4 = g_wC * theta_p_to_SE3(0, sym.Matrix([-w_C, 0, 0]))

g_wJ1 = g_wJ * theta_p_to_SE3(0, sym.Matrix([0, -l_J, 0]))
g_wJ2 = g_wJ * theta_p_to_SE3(0, sym.Matrix([l_J, 0, 0]))
g_wJ3 = g_wJ * theta_p_to_SE3(0, sym.Matrix([0, l_J, 0]))
g_wJ4 = g_wJ * theta_p_to_SE3(0, sym.Matrix([-l_J, 0, 0]))

cup_wall_position_world = [g_wC1, g_wC2, g_wC3, g_wC4]
jack_vertices_world = [g_wJ1, g_wJ2, g_wJ3, g_wJ4]

for i in range(n_cup_walls):

    wall_impact_conditions = []
    wall_impact_equations = []

    for j in range(n_jack_vertices):
        if (i % 2 == 0):
            impact_direction = sym.Matrix([[0, 1, 0, 0]])
        else:
            impact_direction = sym.Matrix([[1, 0, 0, 0]])

        g_wCi = cup_wall_position_world[i]
        g_wJj = jack_vertices_world[j]

        g_CiJj = SE3_inv(g_wCi) * g_wJj

        phi = (impact_direction * g_CiJj * (sym.Matrix([[0, 0, 0, 1]]).T))[0]
        print(f'\033[1mImpact condition of vertex {j + 1} of jack on wall {i + 1}:')
        display(sym.simplify(phi))
        print("\n")

        lambdified_phi = sym.lambdify(sym.Matrix([q]), sub_constants(sym.simplify(phi)))

        dPhidq = sym.simplify(sym.Matrix([phi]).jacobian(q).T)
        dphidq_minus = dPhidq.subs(subs_minus)

        impact_eqns_lhs = sym.Matrix.vstack(dLdqdot_plus - dLdqdot_minus, sym.Matrix([hamiltonian_plus - hamiltonian_minus]))
        impact_eqns_rhs = sym.Matrix.vstack(lam * dphidq_minus, sym.Matrix([0]))
        impact_eqn = sym.Eq(impact_eqns_lhs, impact_eqns_rhs)
        impact_eqn = sub_constants(impact_eqn)

        wall_impact_conditions.append(lambdified_phi)
        wall_impact_equations.append(impact_eqn)

    impact_conditions.append(wall_impact_conditions)
    impact_equations.append(wall_impact_equations)

```

Impact condition of vertex 1 of jack on wall 1:

$$1.0l_C - 1.0l_J \cos(\theta_C(t) - \theta_J(t)) + 1.0x_C(t) \sin(\theta_C(t)) - 1.0x_J(t) \sin(\theta_C(t)) - 1.0y_C(t) \cos(\theta_C(t)) + 1.0y_J(t) \cos(\theta_C(t))$$

Impact condition of vertex 2 of jack on wall 1:

$$1.0l_C - 1.0l_J \sin(\theta_C(t) - \theta_J(t)) + 1.0x_C(t) \sin(\theta_C(t)) - 1.0x_J(t) \sin(\theta_C(t)) - 1.0y_C(t) \cos(\theta_C(t)) + 1.0y_J(t) \cos(\theta_C(t))$$

Impact condition of vertex 3 of jack on wall 1:

$$1.0l_C + 1.0l_J \cos(\theta_C(t) - \theta_J(t)) + 1.0x_C(t) \sin(\theta_C(t)) - 1.0x_J(t) \sin(\theta_C(t)) - 1.0y_C(t) \cos(\theta_C(t)) + 1.0y_J(t) \cos(\theta_C(t))$$

Impact condition of vertex 4 of jack on wall 1:

$$1.0l_C + 1.0l_J \sin(\theta_C(t) - \theta_J(t)) + 1.0x_C(t) \sin(\theta_C(t)) - 1.0x_J(t) \sin(\theta_C(t)) - 1.0y_C(t) \cos(\theta_C(t)) + 1.0y_J(t) \cos(\theta_C(t))$$

Impact condition of vertex 1 of jack on wall 2:

$$-1.0l_J \sin(\theta_C(t) - \theta_J(t)) - 1.0w_C - 1.0x_C(t) \cos(\theta_C(t)) + 1.0x_J(t) \cos(\theta_C(t)) - 1.0y_C(t) \sin(\theta_C(t)) + 1.0y_J(t) \sin(\theta_C(t))$$

Impact condition of vertex 2 of jack on wall 2:

$$1.0l_J \cos(\theta_C(t) - \theta_J(t)) - 1.0w_C - 1.0x_C(t) \cos(\theta_C(t)) + 1.0x_J(t) \cos(\theta_C(t)) - 1.0y_C(t) \sin(\theta_C(t)) + 1.0y_J(t) \sin(\theta_C(t))$$

Impact condition of vertex 3 of jack on wall 2:

$$1.0l_J \sin(\theta_C(t) - \theta_J(t)) - 1.0w_C - 1.0x_C(t) \cos(\theta_C(t)) + 1.0x_J(t) \cos(\theta_C(t)) - 1.0y_C(t) \sin(\theta_C(t)) + 1.0y_J(t) \sin(\theta_C(t))$$

Impact condition of vertex 4 of jack on wall 2:

$$-1.0l_J \cos(\theta_C(t) - \theta_J(t)) - 1.0w_C - 1.0x_C(t) \cos(\theta_C(t)) + 1.0x_J(t) \cos(\theta_C(t)) - 1.0y_C(t) \sin(\theta_C(t)) + 1.0y_J(t) \sin(\theta_C(t))$$

Impact condition of vertex 1 of jack on wall 3:

$$-1.0l_C - 1.0l_J \cos(\theta_C(t) - \theta_J(t)) + 1.0x_C(t) \sin(\theta_C(t)) - 1.0x_J(t) \sin(\theta_C(t)) - 1.0y_C(t) \cos(\theta_C(t)) + 1.0y_J(t) \cos(\theta_C(t))$$

Impact condition of vertex 2 of jack on wall 3:

$$-1.0l_C - 1.0l_J \sin(\theta_C(t) - \theta_J(t)) + 1.0x_C(t) \sin(\theta_C(t)) - 1.0x_J(t) \sin(\theta_C(t)) - 1.0y_C(t) \cos(\theta_C(t)) + 1.0y_J(t) \cos(\theta_C(t))$$

Impact condition of vertex 3 of jack on wall 3:

$$-1.0l_C + 1.0l_J \cos(\theta_C(t) - \theta_J(t)) + 1.0x_C(t) \sin(\theta_C(t)) - 1.0x_J(t) \sin(\theta_C(t)) - 1.0y_C(t) \cos(\theta_C(t)) + 1.0y_J(t) \cos(\theta_C(t))$$

Impact condition of vertex 4 of jack on wall 3:

$$-1.0l_C + 1.0l_J \sin(\theta_C(t) - \theta_J(t)) + 1.0x_C(t) \sin(\theta_C(t)) - 1.0x_J(t) \sin(\theta_C(t)) - 1.0y_C(t) \cos(\theta_C(t)) + 1.0y_J(t) \cos(\theta_C(t))$$

Impact condition of vertex 1 of jack on wall 4:

$$-1.0l_J \sin(\theta_C(t) - \theta_J(t)) + 1.0w_C - 1.0x_C(t) \cos(\theta_C(t)) + 1.0x_J(t) \cos(\theta_C(t)) - 1.0y_C(t) \sin(\theta_C(t)) + 1.0y_J(t) \sin(\theta_C(t))$$

Impact condition of vertex 2 of jack on wall 4:

$$1.0l_J \cos(\theta_C(t) - \theta_J(t)) + 1.0w_C - 1.0x_C(t) \cos(\theta_C(t)) + 1.0x_J(t) \cos(\theta_C(t)) - 1.0y_C(t) \sin(\theta_C(t)) + 1.0y_J(t) \sin(\theta_C(t))$$

Impact condition of vertex 3 of jack on wall 4:

$$1.0l_J \sin(\theta_C(t) - \theta_J(t)) + 1.0w_C - 1.0x_C(t) \cos(\theta_C(t)) + 1.0x_J(t) \cos(\theta_C(t)) - 1.0y_C(t) \sin(\theta_C(t)) + 1.0y_J(t) \sin(\theta_C(t))$$

Impact condition of vertex 4 of jack on wall 4:

$$-1.0l_J \cos(\theta_C(t) - \theta_J(t)) + 1.0w_C - 1.0x_C(t) \cos(\theta_C(t)) + 1.0x_J(t) \cos(\theta_C(t)) - 1.0y_C(t) \sin(\theta_C(t)) + 1.0y_J(t) \sin(\theta_C(t))$$

Impact Update

```
In [20]: def evaluate_impact_condition(x, t):
        q_k = x[0:6]
        qdot_k = x[6:12]
        t_k = t

        impact_conditions_evaluated = []

        for i in range(n_cup_walls):
            for j in range(n_jack_vertices):
                impact_conditions_evaluated.append(impact_conditions[i][j](*q_k))
```

```

return impact_conditions_evaluated

def check_for_impacts(x, t, prev_impact_conditions_evaluated):
    curr_impact_conditions_evaluated = evaluate_impact_condition(x, t)

    impact_regions = np.argwhere(np.sign(curr_impact_conditions_evaluated) != np.sign(prev_impact_conditions_evaluated))
    impact_regions = [idx[0] for idx in impact_regions]

    return impact_regions

def update_states_after_impact(x, t, impact_regions):

    # NOTE: Toggle between the two impact_region selections
    # for different behavior
    # impact_region = np.random.choice(impact_regions)
    impact_region = impact_regions[0]
    wall_index = int(impact_region / n_cup_walls)
    vertex_index = impact_region % n_jack_vertices
    impact_equation = impact_equations[wall_index][vertex_index]

    subs_states = {
        x_C_minus:x[0],
        y_C_minus:x[1],
        theta_C_minus:x[2],
        x_J_minus:x[3],
        y_J_minus:x[4],
        theta_J_minus:x[5],
        x_dot_C_minus:x[6],
        y_dot_C_minus:x[7],
        theta_dot_C_minus:x[8],
        x_dot_J_minus:x[9],
        y_dot_J_minus:x[10],
        theta_dot_J_minus:x[11]
    }

    impact_equation = impact_equation.subs(subs_states)

    impact_variables = sym.Matrix([x_dot_C_plus, y_dot_C_plus, theta_dot_C_plus, x_dot_J_plus, y_dot_J_plus, theta_dot_J_plus, lam])
    impact_solns = sym.solve(impact_equation, impact_variables, dict=True)

    desired_soln = next(soln_ for soln_ in impact_solns if not np.isclose(float(soln_[lam]), 0))
    # display(desired_soln)

    updated_states_after_impact = np.array([
        x[0],
        x[1],
        x[2],
        x[3],
        x[4],
        x[5],
        desired_soln[x_dot_C_plus],
        desired_soln[y_dot_C_plus],
        desired_soln[theta_dot_C_plus],
        desired_soln[x_dot_J_plus],
        desired_soln[y_dot_J_plus],
        desired_soln[theta_dot_J_plus]],
        dtype=float)

    return updated_states_after_impact

```

Simulate the dynamics

```

In [21]: def integrate(f, xt, dt, t):
        """
        Perform numerical integration using the 4th-order Runge-Kutta (RK4) method.

        Parameters:
        =====
        f: Python function
            Derivative of the system ( $\dot{x}(t) = f(x(t), t)$ ).
        xt: NumPy array
            Current state vector  $x(t)$ .
        dt: float
            Time step for integration.
        t: float
            Current simulation time.

        Returns:
        =====
        new_xt: NumPy array
            State vector  $x(t+dt)$  after integration.
        """
        k1 = dt * f(xt, t)
        k2 = dt * f(xt+k1/2., t)
        k3 = dt * f(xt+k2/2., t)
        k4 = dt * f(xt+k3, t)
        new_xt = xt + (1/6.) * (k1+2.*k2+2.*k3+k4)
        return new_xt

def simulate(f, x0, tspan, dt, integrate):
    """
    Simulate the trajectory of a dynamical system over a specified time span.

    Parameters:
    =====
    f: Python function
        Derivative of the system ( $\dot{x}(t) = f(x(t), t)$ ).
    x0: NumPy array
        Initial conditions for the system.
    tspan: list
        Time span of simulation [min_time, max_time].
    dt: float
        Time step for numerical integration.
    integrate: Python function
        Numerical integration method.

    Returns:
    =====
    x_traj: NumPy array
        Simulated trajectory of  $x(t)$  with shape (len(x0), num_time_steps).
    """
    N = int((max(tspan)-min(tspan))/dt)
    x = np.copy(x0)
    tvec = np.linspace(min(tspan),max(tspan),N)
    xtraj = np.zeros((len(x0),N))

    # x0[2] = normalize_angles(x0[2])
    # x0[5] = normalize_angles(x0[5])
    xtraj[:,0] = x0

    prev_impact_conditions_evaluated = evaluate_impact_condition(x0, tvec[0])
    impact_detected = False

    for i in range(1,N):
        impact_regions = check_for_impacts(x, tvec[i], prev_impact_conditions_evaluated)

        if impact_regions and not impact_detected:

```

```

if len(impact_regions) > 1:
    print("\033[1mSimultaneous impact detected \033[0m")
else:
    print("\033[1mSingle Impact detected:\033[0m")
# print(impact_regions)
print(f"Time - {tvec[i]}")

for impact_region in impact_regions:
    wall_index = int(impact_region / n_cup_walls)
    vertex_index = impact_region % n_jack_vertices
    print(f'Vertex {vertex_index + 1} of jack on wall {wall_index + 1}:')

x = update_states_after_impact(xtraj[:,i-1], tvec[i-1], impact_regions)
impact_detected = True
else:
    impact_detected = False

xtraj[:,i] = integrate(f, x, dt, tvec[i])
# xtraj[2,i] = normalize_angles(xtraj[2,i])
# xtraj[5,i] = normalize_angles(xtraj[5,i])

x = np.copy(xtraj[:,i])

return xtraj

def jack_in_cup(x, t):
    """
    Computes the state derivatives for the jack in the cup system.

    Parameters:
    =====
    x: NumPy array
        State vector containing the current state:
        - x_C, y_C, theta_C: Position and angular orientation of the cup.
        - x_J, y_J, theta_J: Position and angular orientation of the jack.
        - xdot_C, ydot_C, thetadot_C: Velocities of the cup (linear and angular).
        - xdot_J, ydot_J, thetadot_J: Velocities of the jack (linear and angular).
    t: float
        Current time.

    Returns:
    =====
    A NumPy array of state derivatives:
    - Linear and angular velocities of cup and jack.
    - Linear and angular accelerations of cup and jack:
      [xdot_C, ydot_C, thetadot_C, xdot_J, ydot_J, thetadot_J, xddot_C, yddot_C, thetaddot_C, xddot_J, yddot_J, thetaddot_J].

    """
    q_k = x[0:6].tolist()
    qdot_k = x[6:12].tolist()
    t_k = [t]

    xddot_C_k = state_dynamics_func[0](*(q_k + qdot_k + t_k))
    yddot_C_k = state_dynamics_func[1](*(q_k + qdot_k + t_k))
    thetaddot_C_k = state_dynamics_func[2](*(q_k + qdot_k + t_k))

    xddot_J_k = state_dynamics_func[3](*(q_k + qdot_k + t_k))
    yddot_J_k = state_dynamics_func[4](*(q_k + qdot_k + t_k))
    thetaddot_J_k = state_dynamics_func[5](*(q_k + qdot_k + t_k))

    qddot_k = [xddot_C_k, yddot_C_k, thetaddot_C_k, xddot_J_k, yddot_J_k, thetaddot_J_k]
    # print(f"Input - {(q_k + qdot_k + t_k)}")
    # print(f"Output - {np.array(qdot_k + qddot_k)}")

```

```
return np.array(qdot_k + qddot_k)
```

Simulate the system

```
In [22]: # Parameters of the simulation
t0 = 0
tf = 10
dt = 0.005
N = int((tf - t0)/dt)
tvec = np.linspace(t0, tf, N)

# Initial condition
# State vector [x_C, y_C, theta_C, x_J, y_J, theta_J, xdot_C, ydot_C, thetadot_C, xdot_J, ydot_J, thetadot_J]
x0 = [-0.5, -0.5, -np.pi/10, 0.1, 0.1, np.pi/10, 0, 0, 0, 1, 1, 0]

# Simulate the system
x_traj = simulate(jack_in_cup, x0, [t0, tf], dt, integrate)
```

Single Impact detected:
Time – 0.625312656328164
Vertex 2 of jack on wall 2:
Single Impact detected:
Time – 0.8254127063531765
Vertex 1 of jack on wall 1:
Single Impact detected:
Time – 2.161080540270135
Vertex 3 of jack on wall 1:
Single Impact detected:
Time – 2.3911955977988995
Vertex 4 of jack on wall 2:
Single Impact detected:
Time – 2.576288144072036
Vertex 1 of jack on wall 3:
Single Impact detected:
Time – 2.976488244122061
Vertex 4 of jack on wall 1:
Single Impact detected:
Time – 3.0665332666333165
Vertex 3 of jack on wall 4:
Single Impact detected:
Time – 3.36168084042021
Vertex 1 of jack on wall 3:
Simultaneous impact detected
Time – 3.7718859429714855
Vertex 3 of jack on wall 1:
Vertex 4 of jack on wall 1:
Single Impact detected:
Time – 4.442221110555278
Vertex 4 of jack on wall 4:
Single Impact detected:
Time – 4.512256128064032
Vertex 3 of jack on wall 3:
Single Impact detected:
Time – 5.092546273136568
Vertex 2 of jack on wall 1:
Single Impact detected:
Time – 6.4032016008004
Vertex 2 of jack on wall 1:
Single Impact detected:
Time – 6.413206603301651
Vertex 3 of jack on wall 2:
Single Impact detected:
Time – 7.053526763381691
Vertex 4 of jack on wall 1:
Single Impact detected:
Time – 7.243621810905452
Vertex 1 of jack on wall 1:
Single Impact detected:
Time – 7.948974487243621
Vertex 3 of jack on wall 1:
Single Impact detected:
Time – 8.544272136068034
Vertex 2 of jack on wall 4:
Single Impact detected:
Time – 8.979489744872437
Vertex 4 of jack on wall 1:

Plots

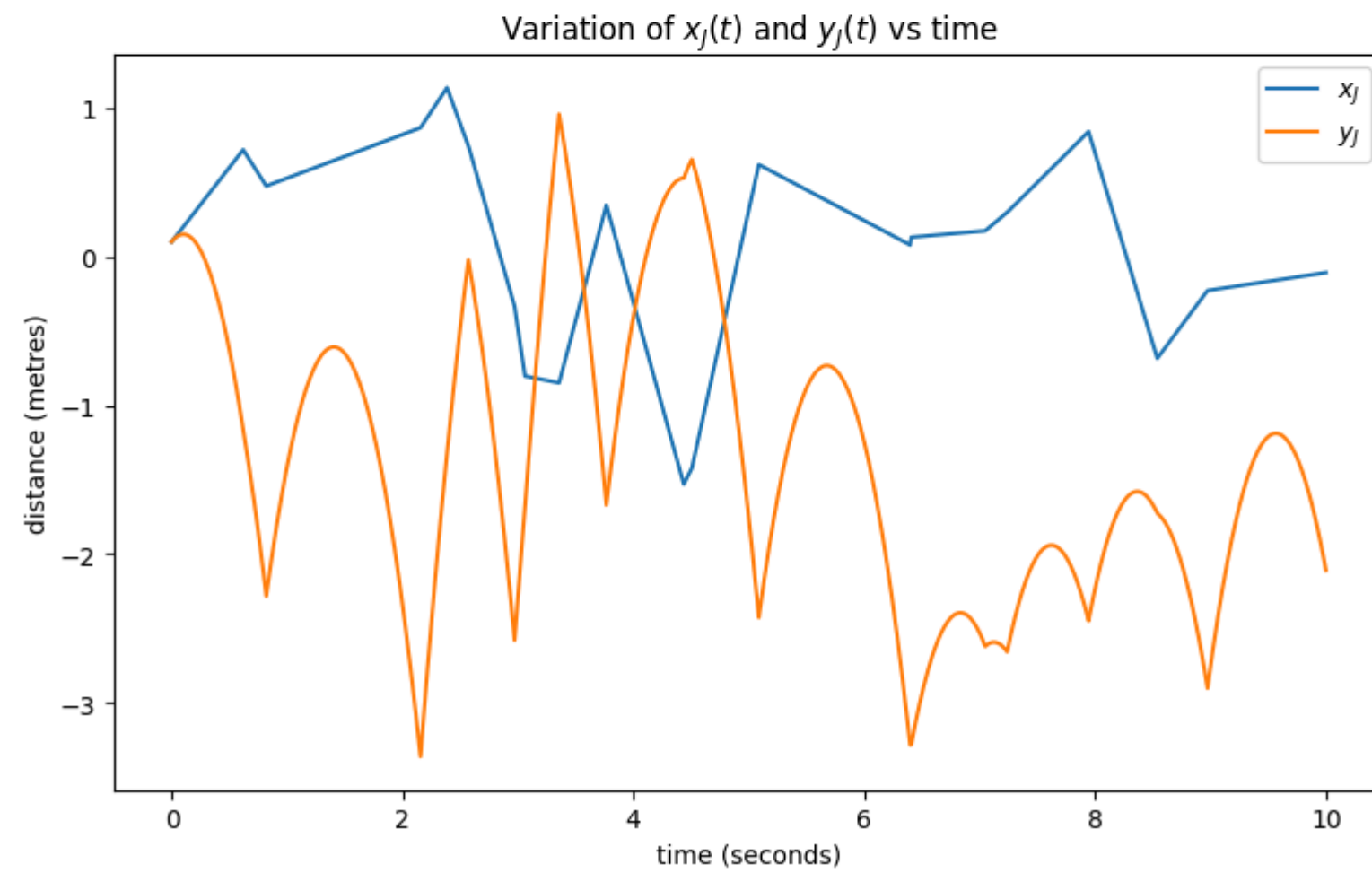
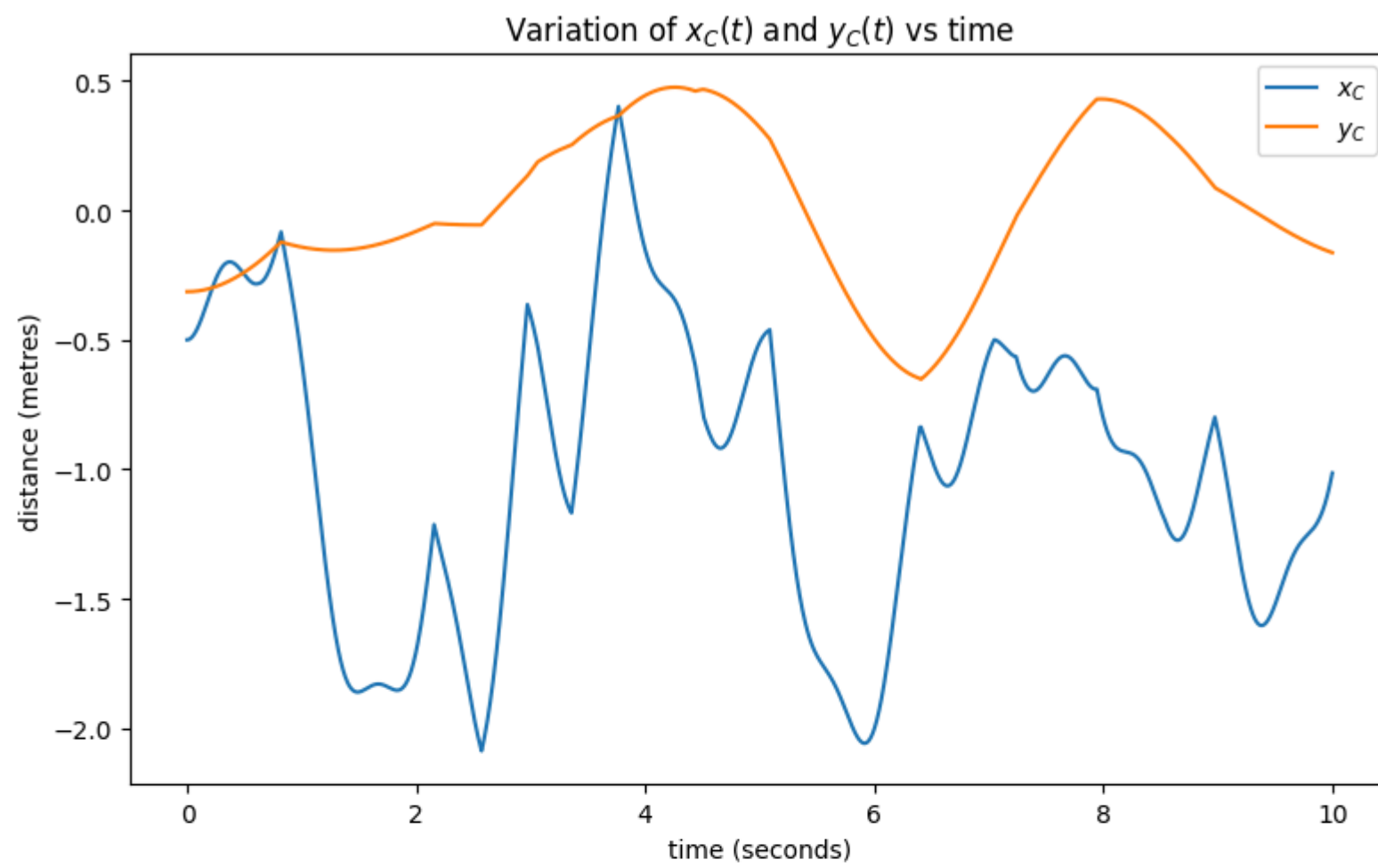
```
In [23]: # Create subplots
fig, axs = plt.subplots(2, 1, figsize=(8, 10)) # 2 rows, 1 column
```



```
# First subplot
axs[0].plot(tvec, x_traj[1, :], label=r"$x_C$")
axs[0].plot(tvec, x_traj[2, :], label=r"$y_C$")
axs[0].set_title(r'Variation of  $x_C(t)$  and  $y_C(t)$  vs time')
axs[0].set_xlabel("time (seconds)")
axs[0].set_ylabel(r"distance (metres)")
axs[0].legend()

# Second subplot
axs[1].plot(tvec, x_traj[3, :], label=r"$x_J$")
axs[1].plot(tvec, x_traj[4, :], label=r"$y_J$")
axs[1].set_title(r'Variation of  $x_J(t)$  and  $y_J(t)$  vs time')
axs[1].set_xlabel("time (seconds)")
axs[1].set_ylabel(r"distance (metres)")
axs[1].legend()

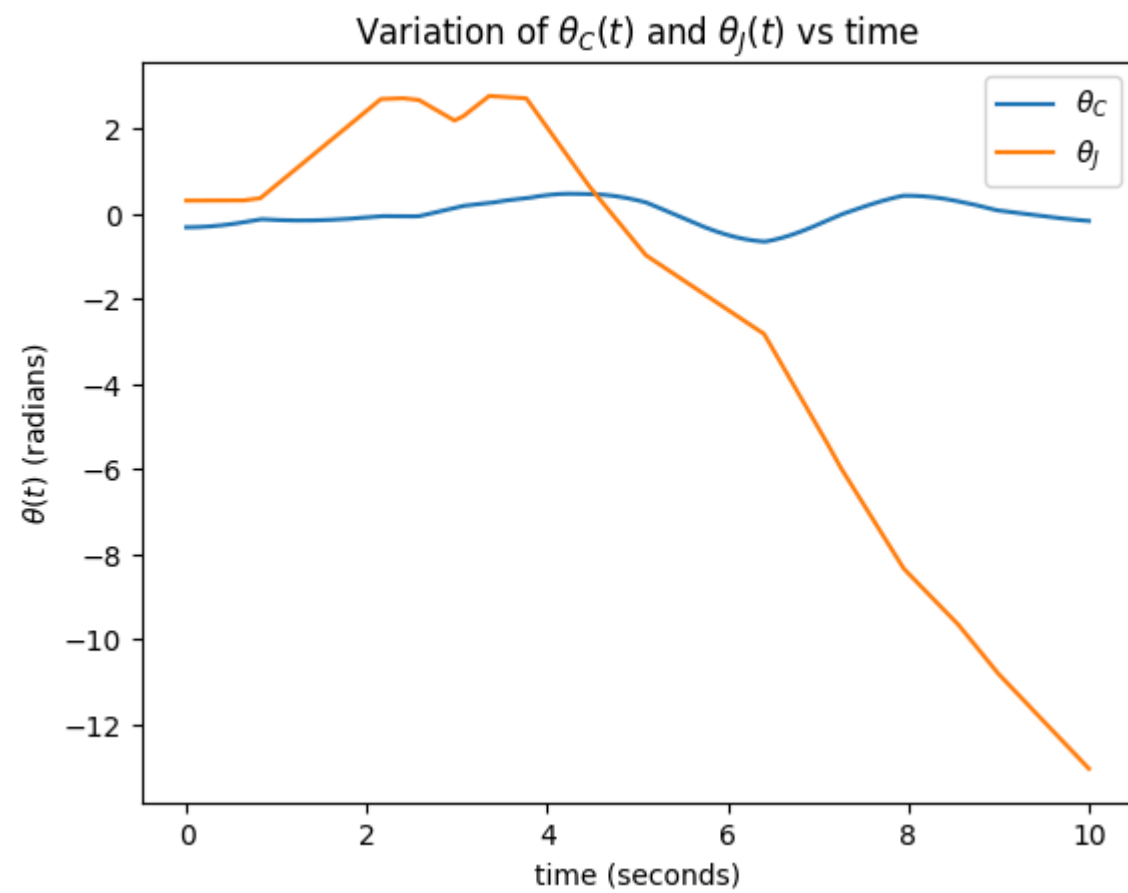
# Adjust layout and display
plt.tight_layout()
plt.show()
```



```
In [24]: # plot
plt.plot(tvec, x_traj[2,:], label=r"$\theta_C$")
plt.plot(tvec, x_traj[5,:], label=r"$\theta_J$")
plt.title(r'Variation of $\theta_C(t)$ and $\theta_J(t)$ vs time')
plt.xlabel("time (seconds)")
plt.ylabel(r"$\theta(t)$ (radians)")
```



```
plt.legend()
plt.show()
```



Animate

```
In [25]: def homogeneous_transform(x):
          return np.concatenate((x, [1]))

def animate_jack_in_cup(state_vect, l_C=3, w_C=1, l_J=1, T=5):
    """
    Function to generate web-based animation of jack in cup

    Parameters:
    =====
    state_vect:
        trajectory of x_C, y_C, theta_C, x_J, y_J, and theta_J
        should be a NumPy array with shape of (6,N)
    L:
        length of the biped leg
    W:
        width of the biped leg
    T:
        length/seconds of animation duration

    Returns: None
    """

    #####
    # Imports required for animation.
    from plotly.offline import init_notebook_mode, iplot
    from IPython.display import display, HTML
    import plotly.graph_objects as go

    #####
    # Browser configuration.
```

```

def configure_plotly_browser_state():
    import IPython
    display(IPython.core.display.HTML('''
        <script src="/static/components/requirejs/require.js"></script>
        <script>
            requirejs.config({
                paths: {
                    base: '/static/base',
                    plotly: 'https://cdn.plot.ly/plotly-1.5.1.min.js?noext',
                },
            });
        </script>
        '''))

configure_plotly_browser_state()
init_notebook_mode(connected=False)

#####
# Getting data from state vector.
x_C = state_vect[0, :]
y_C = state_vect[1, :]
theta_C = state_vect[2, :]

x_J = state_vect[3, :]
y_J = state_vect[4, :]
theta_J = state_vect[5, :]

# Length of simulation
N = len(state_vect[0])

cup = np.zeros((2, 4, N))
cup_corners_x = np.array([w_C, w_C, -w_C, -w_C])
cup_corners_y = np.array([-l_C, l_C, l_C, -l_C])
cup_corners_z = np.zeros(4)

jack = np.zeros((2, 4, N))
jack_vertices_x = np.array([0, l_J, 0, -l_J])
jack_vertices_y = np.array([-l_J, 0, l_J, 0])
jack_vertices_z = np.zeros(4)

for i in range(N):
    # Evaluate homogenous transformation
    g_wC = np.array([
        np.cos(theta_C[i]), -np.sin(theta_C[i]), 0, x_C[i],
        np.sin(theta_C[i]), np.cos(theta_C[i]), 0, y_C[i],
        0, 0, 1, 0],
        [0, 0, 0, 1]
    ])

    g_wJ = np.array([
        np.cos(theta_J[i]), -np.sin(theta_J[i]), 0, x_J[i],
        np.sin(theta_J[i]), np.cos(theta_J[i]), 0, y_J[i],
        0, 0, 1, 0],
        [0, 0, 0, 1]
    ])

    for j in range(4):
        cup[:, j, i] = g_wC.dot(homogeneous_transform([cup_corners_x[j], cup_corners_y[j], cup_corners_z[j]]))[0:2]
        jack[:, j, i] = g_wJ.dot(homogeneous_transform([jack_vertices_x[j], jack_vertices_y[j], jack_vertices_z[j]]))[0:2]

#####
# Using these to specify axis limits.
xm=np.min(cup[0, 3, :])-0.2
xM=np.max(cup[0, 1, :])+0.2
ym=np.min(cup[1, 3, :])-0.2

```

```

yM=np.max(cup[0, 2, :])+0.2

#####
# Defining data dictionary.
# Trajectories are here.
data=[
    dict(name="Cup"),
    dict(name='Jack - Arm1'),
    dict(name='Jack - Arm2'),
    dict(name='Jack - Vertices'),
]

#####
# Preparing simulation layout.
# Title and axis ranges are here.
layout=dict(xaxis=dict(range=[xm, xM], autorange=False, zeroline=False,dtick=1),
            yaxis=dict(range=[ym, yM], autorange=False, zeroline=False,scaleanchor = "x",dtick=1),
            title='Jack in a cup Simulation',
            hovermode='closest',
            updatemenus= [{ 'type': 'buttons',
                            'buttons': [{ 'label': 'Play','method': 'animate',
                                           'args': [None, {'frame': {'duration': T, 'redraw': False}}]},
                                           {'args': [[None], {'frame': {'duration': T, 'redraw': False}, 'mode': 'immediate',
                                           'transition': {'duration': 0}}], 'label': 'Pause','method': 'animate'}
                                ]
                        }
            ])

#####
# Defining the frames of the simulation.
frames=[dict(data=[
    dict(x=[cup[0, 0, k], cup[0, 1, k], cup[0, 2, k], cup[0, 3, k], cup[0, 0, k]],
        y=[cup[1, 0, k], cup[1, 1, k], cup[1, 2, k], cup[1, 3, k], cup[1, 0, k]],
        mode='lines',
        line=dict(color='orange', width=3)
    ),

    dict(x=[jack[0, 0, k], jack[0, 2, k]],
        y=[jack[1, 0, k], jack[1, 2, k]],
        mode='lines',
        line=dict(color='blue', width=3)
    ),

    dict(x=[jack[0, 1, k], jack[0, 3, k]],
        y=[jack[1, 1, k], jack[1, 3, k]],
        mode='lines',
        line=dict(color='blue', width=3)
    ),

    dict(x=[jack[0, 0, k], jack[0, 1, k], jack[0, 2, k], jack[0, 3, k]],
        y=[jack[1, 0, k], jack[1, 1, k], jack[1, 2, k], jack[1, 3, k]],
        mode='markers',
        line=dict(color='purple', width=6)
    ),

]) for k in range(N)]

#####
# Putting it all together and plotting.
figure1=dict(data=data, layout=layout, frames=frames)
iplot(figure1)

```

In [26]: `animate_jack_in_cup(x_traj[0:6,:], l_C = 3, w_C = 2, l_J = 1, T=tf-t0)`

Jack in a cup Simulation

Play

Pause

